Python for AI engineers

Maya Malamud

Class October 2025

Lecture #1

Get to know me

Maya Malamud

10+ years in the Data Science realm

Main experience: NLP & Healthcare

AI consultant | Gen AI lead |Career mentor

אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

האוניברסיטה העברית בירושלים
THE HEBREW UNIVERSITY OF JERUSALEM

amdocs

DataNights

matrix DnA

SPATIALLY HEALTH

משרד הבריאות

WiDS israel

באות

Y-DATA
SCHOOL OF DATA SCIENCE

Class's timeline

17:30 - 18:30	Hour 1
18:30 – 18:45	*Break*
18:45 – 19:45	Hour 2
19:45 – 20:00	*Break*
20:00 – 21:00	Hour 3

Python module - Goals

- Build confidence in programming through hands-on Python practice.
- Learn fundamental programming concepts essential for AI and data science.
- Develop problem-solving and logical thinking skills through practical exercises.
- Gain the ability to use Python for simple data science and automation tasks.

Python module – What you'll learn

- Get motivated: Why Python is the foundation for AI and data science.
- Work in a professional environment with **VS Code** and explore **GitHub Copilot**.
- Master Python basics: syntax, variables, lists, tuples, booleans, and dictionaries.
- Write decision-making code using **if–else**, **loops**, and control flow.
- Understand and use **functions**, **decorators**, and **variable scope**.
- Practice **list comprehensions**, **error handling**, and **regular expressions**.
- Learn the fundamentals of **Object-Oriented Programming (OOP)**.
- Apply everything in a **final Python mini-project** that ties it all together.

Today's agenda

- ✓ (Prerequisites - set up your Python environment)
- ✓ Motivation
- ✓ VS Code - try GitHub Copilot
- ✓ Python syntax & variables
- ✓ Lists

Programming, what is it good for?

Computers are great in performing repetitive tasks:

- A person would write instructions (program) once.
- These instructions may include repetitive operations (e.g. perform a computation on many cases) or can be executed numerous times.

Python’s creation

- Guido van Rossum, a Dutch programmer,
  is the creator of the Python programming language.

- In December 1989, Van Rossum had been looking for
  a "**hobby programming project that would keep him occupied during the week around Christmas**" as his office was closed.

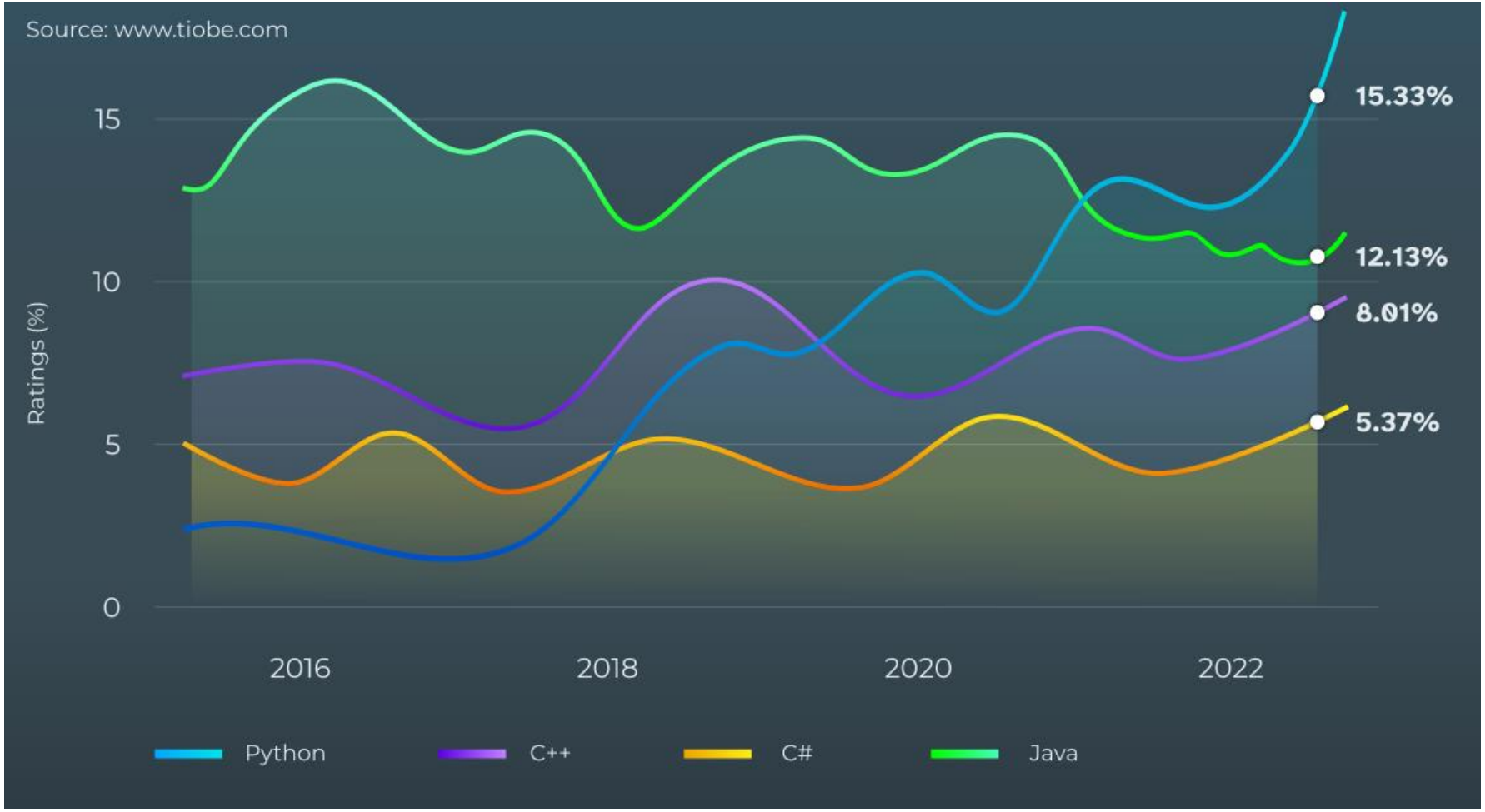
- He attributes choosing the name "Python" to
  "being in a slightly irreverent mood (and a big fan of Monty Python's Flying Circus)”.

Wikipedia

Python’s evolution

Significant updates:

- Python 2.0 (2000)
- Python 3.0 (2008)



Motivation: Why Learn Python for AI?

#1 Python is the #1 Language for AI and Machine Learning

- Every major AI framework is Python-first: **PyTorch**, **TensorFlow**, **Scikit-learn**, **Hugging Face**, **LangChain**, etc.
- All recent breakthroughs in AI (e.g., ChatGPT, AlphaFold, Stable Diffusion) were built and published using Python.
- *"If you're building or using AI models, you're using Python. Period."*

Motivation: Why Learn Python for AI?

#2 Python Makes AI Engineering Fast and Productive

- Minimal syntax: less code to do more.
- Massive ecosystem - libraries for everything:
  - Data wrangling (pandas, numpy)
  - Visualizations (matplotlib, seaborn)
  - Deploying AI models (FastAPI, Gradio)
- Built-in tools for experimentation, reproducibility, and debugging.

Motivation: Why Learn Python for AI?

#3 Designed for Prototyping + Research

- Jupyter notebooks and Python scripts are the standard for exploratory work, especially in research and academia.
- Fast to test hypotheses, tweak models, and run simulations.
- Excellent integration with tools like Git, Copilot, and VS Code.

Motivation: Why Learn Python for AI?

#4 Built by and for Engineers

- Python is a glue code that ties together C++, CUDA, and system-level frameworks.
- Used by NVIDIA, Google, Meta, OpenAI, and in edge AI (e.g., Raspberry Pi, embedded systems).
- Plays well with REST APIs, simulation environments, and hardware control, making it perfect for engineers moving into AI.

Motivation: Why Learn Python for AI?

#5 Interoperability with Existing Skills

- You already understand control flow, data structures, and debugging.

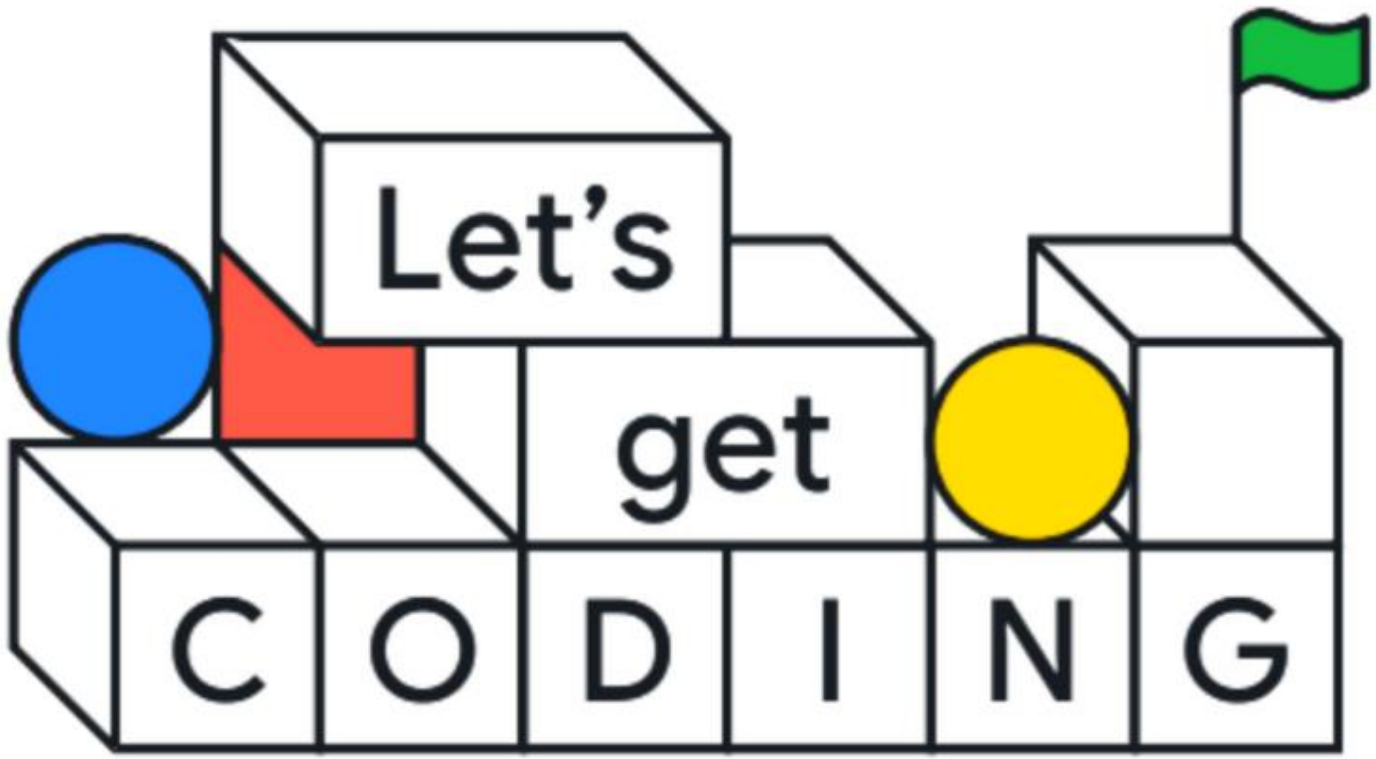
  Python gives you the AI superpowers.

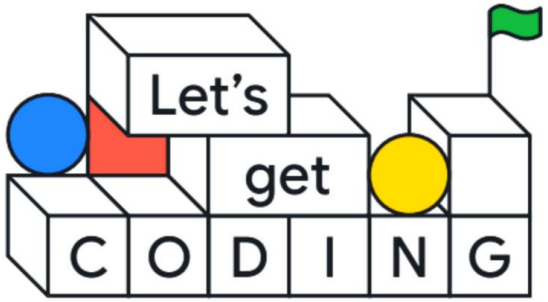
- Python can call C++, use embedded code, or wrap legacy systems.

  You don’t have to throw away what you know!

Installations & tools

Tool	Why?
Python 3.x	Core programming language for the course
VS Code	Code editor with intelligent features + Copilot
Git	Version control tool; required for GitHub Copilot
GitHub Account	Needed to activate GitHub Copilot
GitHub Copilot	AI assistant for fast coding suggestions





💡 Demo – your first VS code

- Open VS Code
- Open folder (create new one)
- Create new file – “hello.py”
- Write a print command:

```
print("Hello, engineers!")
```

- Run the file in the terminal
- Observe your line printed

GitHub Copilot

- A code completion and automatic programming tool

- Predicts code from context and comments
- Helps accelerate learning and development

- Developed by GitHub and OpenAI
- First announced by GitHub on 29 June 2021
- Users can choose the LLM used for generation

GitHub Copilot

💡 **Live Demo**:

Type:

```
# Create a list of even numbers from 1 to 20
```

Let Copilot suggest code.

Run selected cells in VS Code

1. Select one or more lines
- Press Shift + Enter

or
- Right-click and select Run Selection/Line in Interactive Window

or
- Right-click and select Run Selection/Line in Python Terminal

or

2. Turn into cell by typing “# %%” above lines and then Run Cell (will run in Interactive Window)

Python Statements – print()

```
print ("one", "two", "three")
print("four")
print("five")
```

```
one two three
four
five
```

A function can take one or several arguments, separated by commas:

```
print("hi", "there")
```

```
hi there
```

(pay attention to the added space)

Python Statements – print()

Things to notice:

- *print*() must be lower case
  (Function names are case-sensitive)

- No white space at the beginning of the line

- Printed text can be lower/upper case and can have special characters.

Variables: Creating and assigning variables

```
name = "Osnat"
age = 32
height = 1.61
is_student = True

print(f"{name} is {age} years old and {height}m tall.")
print("Hello".lower())
```

```
Osnat is 32 years old and 1.61m tall.
hello
```

```
name = "Osnat"

type(name)
```

```
str
```

Variables: Naming Variables

- Variable name is case sensitive (Z != z)
- Choose meaningful names
- A variable name can consist of:
  - The uppercase letter's "A" through "Z”
  - The lowercase letters "a" through "z"
  - The underscore _
  - Digits (but must not start with digit or contain whitespace)

Variables: reassigning

Variables can be reassigned

```
number = 2
print(number)
number = "hello"
print(number)
```

```
2
hello
```

Data types: int, float

- ***int*** *(*integer*):* *99, 12, -6, 2147652986*
- ***float*** *(*floating *point):* *3.14, 2.5, 6.034e23*

- Special values for int and float: NaN; +/-inf (read more here)
- Multiple assignments at once:

```
x, y = 3, 2
```

Arithmetic operators for int and float

x, y = 3, 2

Operator	Meaning	Description	Example
+	Add	Sum	x + y = 5
-	Subtract	Subtract	x – y = 1
*	Multiply	Multiply	x * y = 6
/	Divide	Divide	x / y =1.5
%	Modulo	Divide and return remainder	3%2=1
**	Exponent	Calculate exponential power value	3**2 = 9
//	Floor Division or Integer Division	Divide and return the integer part	7//2 = 3 and 7.0//2.0 = 3.0

Abbreviated operators

Operator	Example	Same As
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3

Data types

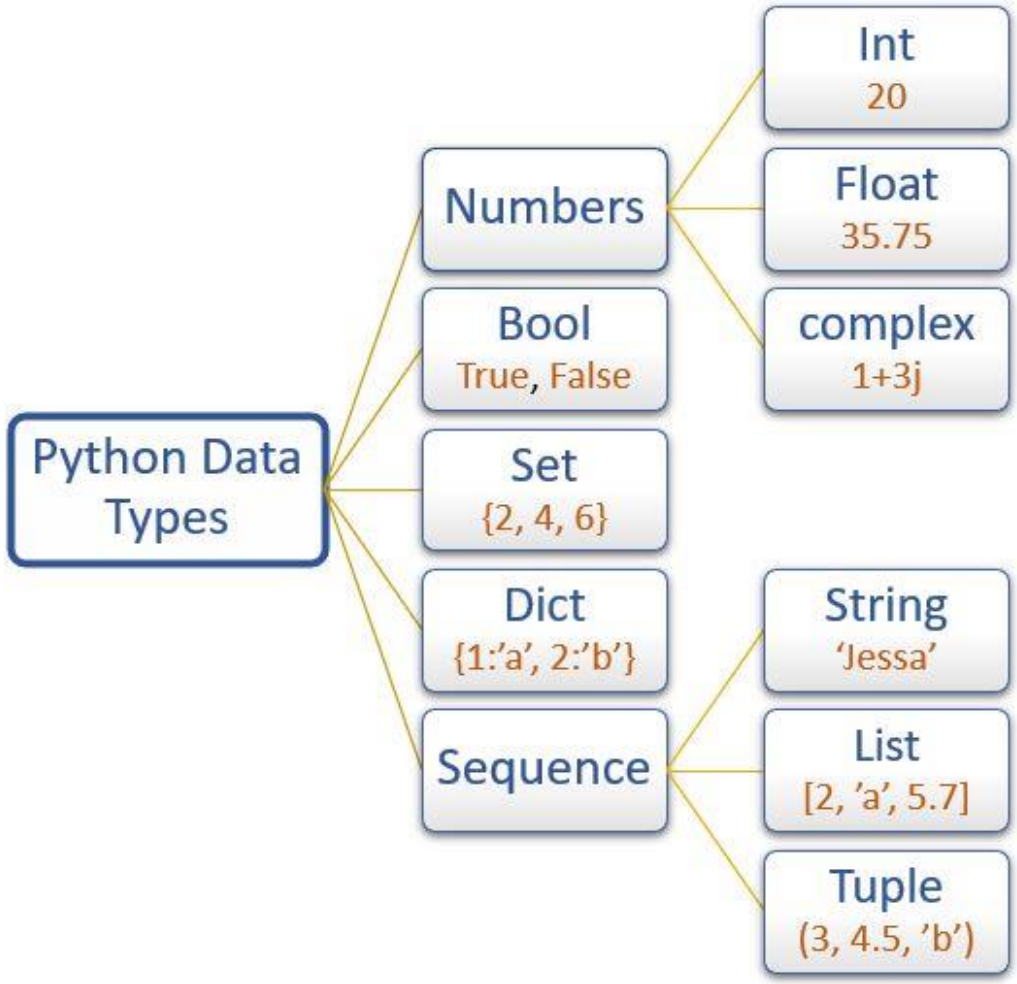
Function type()

```
number = 3
f_number = 2.3
c_number = 3 + 2j
string_s = "Hi from Maya"
is_human = True

print(type(number))
print(type(f_number))
print(type(c_number))
print(type(string_s))
print(type(is_human))
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
<class 'str'>
<class 'bool'>
```

Data types


Python Data Types
Numbers
Int
20
Float
35.75
complex
1+3j
Bool
True, False
Set
{2, 4, 6}
Dict
{1:'a', 2:'b'}
Sequence
String
'Jessa'
List
[2, 'a', 5.7]
Tuple
(3, 4.5, 'b')

Data types: str

- String variables (str) contain text
- To declare a variable as a string, use single / double / triple quotes or add str()

```
Name = "Maya"
name = 'I love Python'
str_number = “3”
str_number2 = str(3)
txt = '''A string in triple quotes can extend
over multiple lines like this one, and can contain
'single' and "double" quotes.'''
```

```
x = 3
y = x * str(x)

print(y)
print(type(y))
```

```
333
<class 'str'>
```

String concatenate operator “+”

Concatenation of strings return a string:

```
str_1 = "Hi"
str_2 = "Maya"
str_combined = str_1 + str_2
print(str_combined)
```

HiMaya

String repetition operator “*”

```
str_1 = "HiMaya "
print(str_1*2)
print(str_1*3)
print(str_1*4)
print(str_1*5)
```

```
HiMaya HiMaya
HiMaya HiMaya HiMaya
HiMaya HiMaya HiMaya HiMaya
HiMaya HiMaya HiMaya HiMaya HiMaya
```

String operators “in” and “not in”

- “a” in the string: Returns boolean True if “a” is in the string and returns False if “a” is not in the string.
- “a” not in the string: Returns boolean True if “a” is not in the string and returns False if “a” is in the string.

```
str1 = "HiMaya "
print("a" in str1)
print("A" in str1)
print("Hi" in str1)
print("Hi" not in str1)
```

```
True
False
True
False
```

String escape sequence operator “\”

An escape character is used to insert a non-allowed character in the given input string: a “\” or “backslash” operator followed by a non-allowed character.

```
# Wrong quotes
print("This class is "Python for AI"")

# Correct quotes: \ allows for double-quotes inside doubled quotes
print("This class is \"Python for AI\"")
```

SyntaxError: invalid syntax

This class is "Python for AI"

Python escape sequences

Escape sequence character	Description	Example	Output
\\	Backslash	>>> print("\\test")	\test
\'	Single-quote	>>> print("Doesn\'t")	Doesn't
\"	Double-quote	>>> print("\"Python\"")	"Python"
\n	New line	print("Python","\n","Lang..")	Python Lang..
\t	Tab	print("Python","\t","Lang..")	Python Lang..

- *We need to use a double **backslash (\\)** to print a **backslash (\),** as \ is an escape character.*
- ***\n** outputs a new line.*

Python escape sequences - example

```
print("C\:Python\number")
```

C\:Python
umber

Python treats \P as an **invalid escape**, so it just keeps it as a backslash + P

```
print("C:\\Python\\number")
```

C:\Python\number

explicitly escaping the backslash (\\)

```
print(r"C:\Python\number")
```

C:\Python\number

The r says to Python: treat it as a raw text, as it is

Additional string operations

Input	Method	Output
"hello world"	capitalize()	Hello world
"hello world"	.isalpha()	False
"123456"	.isnumeric()	True
"hello world"	.isupper()	False
"Hi Alex"	.split()	["Hi", "Alex"]
"hello world"	.title()	Hello World
" Hello "	.strip()	"Hello"
"a b c"	.replace('a', 'd')	"d b c"

"123456".isnumeric() → True

- **lower():** Converts a string to the lowercase letter.
- **upper():** Converts a string to the uppercase letter.
- **islower() / isupper():** Checks if the whole string is in lower case or upper case and returns bool value, respectively.
- **join():** Join different strings to make a single concatenated string. A list of strings is joined together into a single string.
- **split():** Reverse of join(). It splits a large string into smaller strings. By default, a string is split wherever characters such as space, tab or newline characters are found. However, a delimiter can be passed inside the split(). The string then will split when this delimiter is encountered.
- **strip():** Removes white spaces (space, tab and newline) from both sides.
- **len():** Return the length of the string.

String formatting operator “%”

The “%” operator is used to insert another type of variable along with a string

Operator	Description
%d	decimal integer
%s	String
%f	Floating-point real number

```
name = "Osnat"
age = 32
height = 1.61
is_student = True
str1 = "Hey %s!" % (name)
print(str1)

str2 = "Hey %s, you are %d years old and %.2f m tall." % (name, age, height)
print(str2)
```

Hey Osnat!

Hey Osnat, you are 32 years old and 1.61 m tall.

Strings: Indexing

str	f	o	o	b	a	r
index	0	1	2	3	4	5

- Indexing is a way to access individual elements and sequences of objects within your ordered set of data such as string.
- In Python, strings are ordered sequences of character data, and thus can be indexed in this way
- Individual characters in a string can be accessed by specifying the string name followed by a number in square brackets [ ]: str[3] = “b”
- Indexing is zero-based counting: Python start its count from 0!
  So, the first character has index 0, the next has index 1 and so on.
- The index include empty characters such as whitespaces.

Strings: index, count & replace

```
string = "We study Python for AI"
print("Given String:", string)

print('\nString Method index()')
print("Index of 'e' in:'", string, "':", string.index('e'))

print('\nString Method count()')
print("Count of 'o' in'", string, "':", string.count('o'))

print('\nString Method replace()')
print("Replacing 'e' with '3':", string.replace('e', '3'))
```

Given String: We study Python for AI

String Method index()
Index of 'e' in:' We study Python for AI ‘: 1

String Method count()
Count of 'o' in ' We study Python for AI ‘: 2

String Method replace()
Replacing 'e' with '3' : W3 study Python for AI

Strings: Slicing

str	f	o	o	b	a	r
index	0	1	2	3	4	5

- Slicing is indexing syntax that extracts a portion from a string.
- To slice a portion use colon (:) inside the square brackets with the desire indexes.

```
str = "foobar"

# Indexing
print(str[0])
print(str[3])

# Slicing
print(str[0:3])
print(str[:3])
print(str[3:])
```

```
f
b

foo
foo
bar
```

Strings: Slicing with negative indexing

str	f	o	o	b	a	r
Negative index	-6	-5	-4	-3	-2	-1

- Omitting both indices [:] returns the original string
- str[7]: Indexing with a number greater than len(string) will raise an error

```
str = "foobar"

print(str[-1])
print(str[-6:-3])
print(str[2:len(str)-2])

print(str[:])

print(str[0:8]) # out of range
print(str[7]) # out of range
```

```
r
foo
ob
foobar
foobar
IndexError: string index out of range
```

dir() function

The function dir(message) will return all methods of the variable message, essentially all the the methods that can be applied on strings.

```
>>> message = 'Hello World'
>>> dir(message)
['__add__', '__class__', '__contains__', '__delattr__', '__doc__', '__eq__', '__for
mat__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__getslice_
_', '__gt__', '__hash__', '__init__', '__le__', '__len__', '__lt__', '__mod__', '__
mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__',
 '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '_formatte
r_field_name_split', '_formatter_parser', 'capitalize', 'center', 'count', 'decode'
, 'encode', 'endswith', 'expandtabs', 'find', 'format', 'index', 'isalnum', 'isalph
a', 'isdigit', 'islower', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower'
, 'lstrip', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rspl
it', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', '
translate', 'upper', 'zfill']
```

help() function

The function help() will return what does the method do.

For example: help(message.count)

```
In [6]: help(message.count)
Help on built-in function count:

count(...) method of builtins.str instance
    S.count(sub[, start[, end]]) -> int

    Return the number of non-overlapping occurrences of substring sub in
    string S[start:end].  Optional arguments start and end are
    interpreted as in slice notation.
```

Python reserved words (keywords)

Don’t use them for naming variables.

and	except	lambda	with
as	finally	nonlocal	while
assert	false	None	yield
break	for	not	
class	from	or	
continue	global	pass	
def	if	raise	
del	import	return	
elif	in	True	
else	is	try	

```
import keyword
print(keyword.kwlist)
```

Data types: list

A list is a collection of things, enclosed in [ ] and separated by commas.

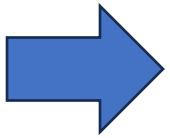
```
List_1 = [10, 20, 14]
List_2 = ["Geeks", "For", "Geeks"]
```

Data types: list
Creating a list

```
# Creating a List
List = []
print("Blank List: ")
print(List)

# Creating a List of numbers
List = [10, 20, 14]
print("\nList of numbers: ")
print(List)

# Creating a List of strings and accessing
# using index
List = ["Geeks", "For", "Geeks"]
print("\nList Items: ")
print(List[0])
print(List[2])
```



Output

```
Blank List:
[]

List of numbers:
[10, 20, 14]

List Items:
Geeks
Geeks
```

Data types: list
Creating a list – cont.

```
# Creating a List with
# the use of Numbers
# (Having duplicate values)
List = [1, 2, 4, 4, 3, 3, 3, 6, 5]
print("\nList with the use of Numbers: ")
print(List)

# Creating a List with
# mixed type of values
# (Having numbers and strings)
List = [1, 2, 'Geeks', 4, 'For', 6, 'Geeks']
print("\nList with the use of Mixed Values: ")
print(List)
```

Output

```
List with the use of Numbers:
[1, 2, 4, 4, 3, 3, 3, 6, 5]

List with the use of Mixed Values:
[1, 2, 'Geeks', 4, 'For', 6, 'Geeks']
```

Data types: list
Creating a list using split()

```
str1 = "We study Python for AI"
lst1 = str1.split()
print(lst1)
print(type(lst1))

lst2 = str1.split("o")
print(lst2)
lst3 = str1.split(" ")
print(lst3)
lst4 = str1.split(" ", 2)
print(lst4)
```

```
['We', 'study', 'Python', 'for', 'AI']
<class 'list'>
['We study Pyth', 'n f', 'r AI']
['We', 'study', 'Python', 'for', 'AI']
['We', 'study', 'Python for AI']
```

Several consecutive whitespaces:

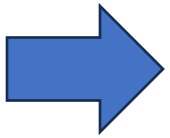
- .split() → will take consecutive whitespaces as one
- .split(" ") → will take a whitespace between other whitespaces as a string of whitespace " "

Data types: list
Accessing an element in a list

```
# Python program to demonstrate
# accessing of element from list

# Creating a List with
# the use of multiple values
List = ["Geeks", "For", "Geeks"]

# accessing a element from the
# list using index number
print("Accessing a element from the list")
print(List[0])
print(List[2])
```



Output

```
Accessing a element from the list
Geeks
Geeks
```

Data types: list
Creating & accessing a multi-dimensional / nested list

```
# Creating a Multi-Dimensional List
# (By Nesting a list inside a List)
List = [['Geeks', 'For'], ['Geeks']]

# accessing an element from the
# Multi-Dimensional List using
# index number
print("Accessing a element from a Multi-Dimensional list")
print(List[0][1])
print(List[1][0])
```

Output

```
Accessing a element from a Multi-Dimensional list
For
Geeks
```

Data types: list
Accessing an element in a list: Negative indexing

```
List = [1, 2, 'Geeks', 4, 'For', 6, 'Geeks']

# accessing an element using
# negative indexing
print("Accessing element using negative indexing")

# print the last element of list
print(List[-1])

# print the third last element of list
print(List[-3])
```

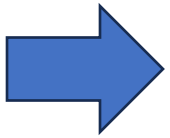
Output

```
Accessing element using negative indexing
Geeks
For
```

Data types: list
Getting the size of a list: len()

```
# Creating a List
List1 = []
print(len(List1))

# Creating a List of numbers
List2 = [10, 20, 14]
print(len(List2))
```



Output

```
0
3
```

Data types: list
Taking input for a list

We can take the input of a list of elements as string, integer, float, etc. But the default one is a string.

```
# Python program to take space
# separated input as a string
# split and store it to a list
# and print the string list

# input the list as string
string = input("Enter elements (Space-Separated): ")

# split the strings and store it to a list
lst = string.split()
print('The list is:', lst)    # printing the list
```

Output:

```
Enter elements: GEEKS FOR GEEKS
The list is: ['GEEKS', 'FOR', 'GEEKS']
```

Data types: list
Adding elements to a list: append()

```
# Creating a List
List = []
print("Initial blank List: ")
print(List)

# Addition of Elements
# in the List
List.append(1)
List.append(2)
List.append(4)
print("\nList after Addition of Three elements: ")
print(List)


# Addition of List to a List
List2 = ['For', 'Geeks']
List.append(List2)
print("\nList after Addition of a List: ")
print(List)
```

Output

Initial blank List:
[]

List after Addition of Three elements:
[1, 2, 4]

List after Addition of a List:
[1, 2, 4, ['For', 'Geeks']]

Data types: list
Adding elements to a list: insert()

```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = [1,2,3,4]
print("Initial List: ")
print(List)

# Addition of Element at
# specific Position
# (using Insert Method)
List.insert(3, 12)
List.insert(0, 'Geeks')
print("\nList after performing Insert Operation: ")
print(List)
```

Output

```
Initial List:
[1, 2, 3, 4]

List after performing Insert Operation:
['Geeks', 1, 2, 3, 12, 4]
```

Data types: list
Adding elements to a list: extend()

```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = [1, 2, 3, 4]
print("Initial List: ")
print(List)

# Addition of multiple elements
# to the List at the end
# (using Extend Method)
List.extend([8, 'Geeks', 'Always'])
print("\nList after performing Extend Operation: ")
print(List)
```

Output

```
Initial List:
[1, 2, 3, 4]

List after performing Extend Operation:
[1, 2, 3, 4, 8, 'Geeks', 'Always']
```

Data types: list
Reversing a list

```
# Reversing a list
mylist = [1, 2, 3, 4, 5, 'Geek', 'Python']
mylist.reverse()
print(mylist)
```

Output

```
['Python', 'Geek', 5, 4, 3, 2, 1]
```

Data types: list
Removing elements from a list: remove()

```
# Python program to demonstrate
# Removal of elements in a List

# Creating a List
List = [1, 2, 3, 4, 5, 6,
        7, 8, 9, 10, 11, 12]
print("Initial List: ")
print(List)

# Removing elements from List
# using Remove() method
List.remove(5)
List.remove(6)
print("\nList after Removal of two elements: ")
print(List)
```

Output

```
Initial List:
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

List after Removal of two elements:
[1, 2, 3, 4, 7, 8, 9, 10, 11, 12]
```

Data types: list
Removing elements from a list: pop()

```
List = [1, 2, 3, 4, 5]

# Removing element from the
# Set using the pop() method
List.pop()
print("\nList after popping an element: ")
print(List)

# Removing element at a
# specific location from the
# Set using the pop() method
List.pop(2)
print("\nList after popping a specific element: ")
print(List)
```

Output

```
List after popping an element:
[1, 2, 3, 4]

List after popping a specific element:
[1, 2, 4]
```

```
a = [10, 20, 30, 40]

# Remove and save the last element from list a
val = a.pop()

print(val)
print(a)
```

```
40
[10, 20, 30]
```

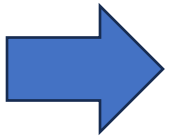
Data types: list
Slicing a list to extract data

```
# Creating a List
List = ['G', 'E', 'E', 'K', 'S', 'F',
        'O', 'R', 'G', 'E', 'E', 'K', 'S']
print("Initial List: ")
print(List)

# Print elements of a range
# using Slice operation
Sliced_List = List[3:8]
print("\nSlicing elements in a range 3-8: ")
print(Sliced_List)

# Print elements from a
# pre-defined point to end
Sliced_List = List[5:]
print("\nElements sliced from 5th "
      "element till the end: ")
print(Sliced_List)

# Printing elements from
# beginning till end
Sliced_List = List[:]
print("\nPrinting all elements using slice operation: ")
print(Sliced List)
```



Output

```
Initial List:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']

Slicing elements in a range 3-8:
['K', 'S', 'F', 'O', 'R']

Elements sliced from 5th element till the end:
['F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']

Printing all elements using slice operation:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']
```

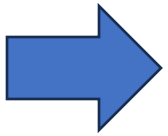
Data types: list
Slicing a list to extract data (negative indexing)

```
# Creating a List
List = ['G', 'E', 'E', 'K', 'S', 'F',
        'O', 'R', 'G', 'E', 'E', 'K', 'S']
print("Initial List: ")
print(List)

# Print elements from beginning
# to a pre-defined point using Slice
Sliced_List = List[:-6]
print("\nElements sliced till 6th element from last: ")
print(Sliced_List)

# Print elements of a range
# using negative index List slicing
Sliced_List = List[-6:-1]
print("\nElements sliced from index -6 to -1")
print(Sliced_List)

# Printing elements in reverse
# using Slice operation
Sliced_List = List[::-1]
print("\nPrinting List in reverse: ")
print(Sliced_List)
```



Output

```
Initial List:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']

Elements sliced till 6th element from last:
['G', 'E', 'E', 'K', 'S', 'F', 'O']

Elements sliced from index -6 to -1
['R', 'G', 'E', 'E', 'K']

Printing List in reverse:
['S', 'K', 'E', 'E', 'G', 'R', 'O', 'F', 'S', 'K', 'E', 'E', 'G']
```

Data types: list
”+” and “*” operators on lists

```
In [2]: lst1 = [1,2,3]

In [3]: lst2 = [4,5]

In [4]: lst1+lst2
Out[4]: [1, 2, 3, 4, 5]


In [5]: 4*lst1
Out[5]: [1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Data types: list
List Methods

Method	Description
List.append(x)	Add item to the end, same as a[len(a):] = [x]
List.extend(list2)	Appends the contents of a list to a list
List.pop(x)	Removes and returns the last item in the list
List.insert(index, x)	Inserts object x into a list at an index
X in list	Check if an item exists in the list
List.count(x)	Returns the number of times x appears in the list
List.reverse()	Reverses the elements of the list, in place.
List.remove(x)	Removes the first item from the list whose value is x, It gives an error if there is no such item.
List.index(x)	Returns the index in the list of the first item whose value is x. It gives an error if there is no such item.
List.sort()	Sorts objects of list

Class exercise – Lecture 1

- Create a list of names, such as:

  [”Amit", “Sharon”, ”Dana”, ”Sarah”, “Tal”]

- Slice the list to keep only the first 3 people.
- Use indexing to remove the second person.
- Add a new person at the end.

Class exercise – suggested solution

```python
# 1. Create a list of names
names = ["Amit", "Sharon", "Dana", "Sarah", "Tal"]

# 2. Slice the list to keep only the first 3 people
names = names[:3]  # Now: ["Amit", "Sharon", "Dana"]

# 3. Use indexing to remove the second person
del names[1]  # Removes "Sharon", now: ["Amit", "Dana"]

# 4. Add a new person at the end
names.append("Noa")  # Now: ["Amit", "Dana", "Noa"]

# Final output
print(names)
```

['Amit', 'Dana', 'Noa']

Additional recommended resources

- Python:
  - Python for beginners link
  - Learn Python with VS code link
  - DataCamp link
  - Python-course link
  - Google's Python Class link
- Copilot:
  - GeekAcademy - GitHub Copilot in VS Code link
  - Prompting for GitHub Copilot In VS Code link
- The Missing Semester of Your CS Education link